

ShorlyNot: Quantum-Inspired Tamper-Evidence and Incident Response for Banking

Team ShorlyNot

Smart India Hackathon 2026 Problem Statement 26141 (Egreen Quanta)

Contents

1	Executive Summary	2
2	Introduction	2
3	Objectives	3
4	System Overview	3
5	Verification Using QDS-T1	4
5.1	Quantum Signature Generation	4
5.2	Verification and Mismatch Rate	4
5.3	Acceptance Threshold τ	5
5.4	Forgery Bound P_{forge}	6
5.5	Message Binding	7
6	Channel and Session Security	8
6.1	AES-GCM Wrap	8
6.2	BB84 QBER Estimation	8
6.3	Replay Protection	8
6.4	Trust Boundaries	8
7	Threat Classification and Response	9
8	Monitoring and Dashboard	10
9	Demo Walkthrough	11
9.1	Case Study 1: Honest Transfer	11
9.2	Case Study 2: Forgery and Account Quarantine	11
9.3	Case Study 3: Channel Compromise and Full Lock	11
10	Conclusion	11
11	Future Work	12
	References	12

1 Executive Summary

Banking depends on digital signatures to approve transactions. Classical signatures (RSA, Ed25519, ML-DSA) answer one question: who signed. They do not tell a bank what to do when a signature looks wrong right now. ShorlyNot is a software framework that verifies teleportation-style quantum digital signatures (QDS), measures the mismatch between a claimed signature and the expected one, names the threat, and locks the affected account or the whole bank according to severity.

What the current system provides is graded tamper-evidence: it checks whether a claimed signature still matches the evidence captured at signing time and responds in stages. It is not a transferable signature scheme; non-repudiation and dispute adjudication are out of scope and listed in Future Work. The PS asks for a detection and response framework, and that is what this is.

Three numbers describe the protocol-level guarantees under the unentangled-adversary model. The acceptance threshold $\tau = 0.2097$ is derived from Hoeffding’s inequality with noise floor $p_0 = 0.02$, false-reject budget $\delta = 0.01$ and $n = 64$ check positions. A forger without the entangled resource passes with probability $P_{\text{forge}} = 9.4048 \times 10^{-7}$ at $n = 64$, falling to 7.78×10^{-17} at $n = 128$. Any change to the transaction after signing flips about half of the expected check bits and, under the SHA-256 avalanche model stated in Section 5.5, is caught with probability at least $1 - P_{\text{forge}}$. Section 5 derives these values step by step. Both numbers are scoped to that named model; they say nothing about an adversary who can read the classical data stores, which Section 6 treats separately.

ShorlyNot runs on Qiskit Aer, which is a classical simulator. The project does not claim physical quantum hardware or information-theoretic security from hardware. To say it plainly: what this deployment provides is a correctly simulated statistical model of a teleportation-based QDS protocol, where the operational security work in the current build is done by the classical layers, namely SHA-256, AES-256-GCM, HKDF, and access control on the data stores. The quantum layer is load-bearing for the PS deliverable (the protocol shape, the mismatch statistics, the threat ladder) and descriptive for the deployment (Section 6 states those boundaries plainly). The protocol follows the PS-mandated structure of the QDS literature (teleportation, Pauli corrections, statistical thresholds) but does not reproduce the multi-verifier, unclonability-based security arguments of Gottesman-Chuang (2001) or Wallden et al. (2015); those remain the reference model, and Section 2 states exactly what is borrowed and what is not. All numbers quoted in this document are computed or measured by repository scripts (`scripts/scoreboard.py` and `analysis/benchmark.py`) and match the code; the one empirical cell that varies between runs is disclosed in Section 8.

2 Introduction

Quantum digital signatures were introduced by Gottesman and Chuang in 2001 and extended for quantum key distribution components by Wallden, Dunjko, Kent and Andersson in 2015. Problem Statement 26141 asks for this family of protocols: teleportation-style QDS verification, Pauli and statistical thresholds, a non-ML threat detection ladder, a working framework, and a dashboard with logs for banking use. The QDS-T1¹ verification pattern used here is inspired by the keyword-level structure of those protocols (state preparation, Bell measurement, mismatch statistics), but runs on a simulator and provides graded tamper-evidence rather than transferable signatures.

ShorlyNot is built as two products:

- **skeleton/** is the QDS-T1 engine. It signs and verifies with Qiskit Aer teleportation circuits, computes the Hoeffding threshold, runs the Q-STDF threat classifier, and maintains the S0 to S4 stage machine.
- **bank/** is a mock bank with a SOC dashboard. It consumes the engine over a REST API and enforces lockdown at the application layer. No iptables, no hardware assumptions.

¹The label QDS-T1 is retained from the problem statement’s quantum-digital-signature terminology; as this section explains, the construction provides tamper-evidence, not a signature scheme in the classical or QDS sense.

Relationship to prior work. What is borrowed from the QDS literature is the BB84-style channel error model behind the QBER estimate, the teleportation circuit semantics with Pauli corrections, and the use of statistical thresholds in place of bit equality. What is not reproduced is the multi-verifier signing structure and the unclonability-based security arguments of Gottesman-Chuang and Wallden et al. This implementation has a single verifier, the bank, which holds the expected syndromes in its session store. Verification is therefore a symmetric secret-comparison construction, closer in shape to a keyed fuzzy match than to a public-key signature, which is why this document calls the guarantee tamper-evidence and lists non-repudiation under Future Work.

The full demo takes about two minutes. An honest transfer verifies and commits. A forged bundle is rejected by mismatch statistics alone. The SOC names the threat class, quarantines the affected account, and escalates to a full bank lock only when the evidence is sustained or critical. The remaining sections explain the mathematics behind each step of that path.

3 Objectives

1. Specify the sign and verify procedures of the QDS-T1 protocol.
2. Derive the acceptance threshold τ from Hoeffding’s inequality and show that an honest signature is rejected with probability at most δ .
3. Derive the forgery bound P_{forge} and show it decays exponentially in the number of check positions n .
4. Describe the supporting checks: payload binding, replay protection, channel integrity, and the escalation policy.
5. Report measured benchmarks and state clearly what is simulated and what is not.

4 System Overview

Each transaction flows through seven steps inside the pipeline (`pipeline.py` in the skeleton package):

1. A BB84 session negotiates a symmetric key and estimates the QBER.
2. The transaction is hashed (SHA-256) and 64 check positions are derived from the hash.
3. A Qiskit Aer circuit runs the teleportation protocol for each check position and records the Bell measurement outcomes as the syndrome.
4. The syndrome bits are wrapped with AES-256-GCM under the session key.
5. The verifier reconstructs the expected state from the Bob-side session store, applies the claimed syndrome, and counts mismatches.
6. The Q-STDF ladder classifies the transfer: OK, forgery, impersonation, replay, channel, parameter tampering, or unauthorized verifier.
7. The stage machine enforces the response: nothing, account quarantine, or full bank lock.

The bank never touches cryptography. It only consumes the verdict and enforces it. This keeps the security logic in one place and makes the dashboard honest: everything the SOC shows comes from the engine.

5 Verification Using QDS-T1

5.1 Quantum Signature Generation

The encoder maps a classical bit b and a basis β (0 for Z, 1 for X) to a Pauli eigenstate:

$$b=0, \beta=0 : |0\rangle \quad b=1, \beta=0 : |1\rangle \quad b=0, \beta=1 : |+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}} \quad b=1, \beta=1 : |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}} \quad (1)$$

Alice shares a Bell pair with Bob, $|\Phi^+\rangle_{12} = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$. The signing circuit for one check position runs on Qiskit Aer [11] (`engines/qiskit_aer.py`):

1. Prepare the message state on qubit 0 and the Bell pair on qubits 1 and 2.
2. Apply CNOT from qubit 0 to qubit 1, then Hadamard on qubit 0.
3. Measure qubits 0 and 1. The two bits (m_1, m_2) are the syndrome for this check position.
4. Bob would recover the message by applying $X^{m_2}Z^{m_1}$ to qubit 2.

Each Bell measurement produces a two-bit classical outcome. The outcome is uniformly distributed, so it carries two bits of entropy; by itself it reveals no information about the teleported state. A signature over n positions carries $2n$ syndrome bits. The signer stores the ground truth (nonce, key id, payload hash, syndromes) in the Bob-side session store. This store plays the role of Bob's half of the Bell pair, which is the physical ground truth in a real implementation. The syndromes are uniformly random per signing; they do not encode the payload. Their evidence value comes entirely from the store keeping the ground truth, and secrecy of that store is therefore the binding constraint (Section 6.4). Because a single verifier holds the ground truth, the scheme is symmetric: it detects substitution of the claimed syndromes, and it does not provide public verifiability or non-repudiation.

5.2 Verification and Mismatch Rate

For each check position the verifier:

1. Recomputes the expected bit from the payload hash.
2. Takes the claimed syndrome from the bundle and the true syndrome from the session store.
3. Computes the state Bob would hold: apply the true correction in reverse, then the claimed correction.
4. Measures in the chosen basis under a depolarizing noise floor $p_0 = 0.02$.
5. Counts the mismatch if the measured bit differs from the expected bit.

Verification never re-executes a circuit or performs a fresh Bell measurement at check time; the mismatch test is an analytic Born-rule evaluation over the claimed and stored syndromes, and the randomness in the check is statistical rather than a physical measurement.

The mismatch rate is the average over all positions:

$$\hat{p} = \frac{\text{mismatches}}{n} \quad (2)$$

Where:

- $\hat{p}$ = observed mismatch rate

- n = number of check positions (64 in the default preset)

For an honest signature the verifier applies the correct syndrome, so the message state is recovered. Under the noise model the per-check error is $p_0/2 = 0.01$. A forger has no access to the session store. Her claimed syndromes are unrelated to the true ones, so each check behaves like a fair coin and the mismatch rate sits near 0.5. That gap between 0.01 and 0.5 is the detection signal.

5.3 Acceptance Threshold τ

The threshold comes from Hoeffding’s inequality (Hoeffding, 1963). For independent variables in $[0, 1]$ with mean μ and any $t > 0$:

$$P(\bar{X} - \mu \geq t) \leq e^{-2nt^2} \quad (3)$$

The bound is centered on the noise floor $p_0 = 0.02$, not on the honest per-check mean $p_0/2 = 0.01$ from the previous section. That is a deliberately padded baseline: the true mean sits below the center, which leaves headroom for unmodeled noise, at the cost of a higher forgery pass line (the trade-off is quantified after the example). Setting the false-reject probability equal to the budget δ and solving for τ gives:

$$\tau = p_0 + \sqrt{\frac{\ln(1/\delta)}{2n}} \quad (4)$$

Where:

- p_0 = noise floor (0.02)
- δ = false-reject budget (0.01 in the NORMAL preset)
- n = number of check positions (64)

The honest-side variable is Bernoulli, so an exact binomial quantile would give a tighter threshold than the Hoeffding form. The Hoeffding bound is kept because it is closed-form, and its padding is deliberate, as quantified below.

Example Calculation:

- Input: $p_0 = 0.02$, $\delta = 0.01$, $n = 64$.
- Margin: $\sqrt{\ln(100)/128} = \sqrt{4.60517/128} = 0.18968$.
- Threshold: $\tau = 0.02 + 0.18968 = 0.2097$.
- Honest reject probability stays below $\delta = 0.01$ by construction.

The verifier accepts the signature when $\hat{p} \leq \tau$. The threshold is built on the noise floor $p_0 = 0.02$, not on the honest per-check error $p_0/2 = 0.01$ derived in the previous section. This is a deliberate trade and it cuts both ways. On the false-reject side it adds margin: the true mean sits below the bound, so the Hoeffding guarantee holds with room to spare. On the forgery side a higher threshold raises the forger’s pass line, because P_{forge} sums the binomial tail up to $\lfloor n\tau \rfloor$. The table below shows both choices side by side, computed with the exact binomial CDF:

n	τ as built ($p_0=0.02$)	τ at mean 0.01	P_{forge} as built	P_{forge} tightened	Cost
32	0.2882	0.2782	1.0031×10^{-2}	3.500×10^{-3}	$2.9 \times$
64	0.2097	0.1997	9.4048×10^{-7}	2.283×10^{-7}	$4.1 \times$
128	0.1541	0.1441	7.7792×10^{-17}	1.327×10^{-17}	$5.9 \times$

The chosen threshold keeps P_{forge} near one in a million at $n = 64$, which is more than sufficient for the threat model here, and it matches every benchmark, dashboard and slide in the repository. A deployment that wants the tighter bound can re-parameterize the model; the table states what that buys. The presets are STRICT ($\delta = 0.001$, $\tau = 0.2523$), NORMAL ($\delta = 0.01$, $\tau = 0.2097$) and LENIENT ($\delta = 0.05$, $\tau = 0.1730$).

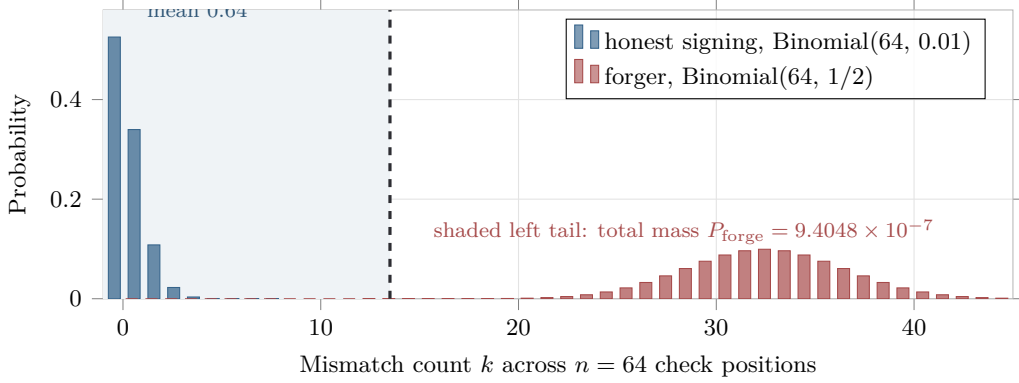


Figure 1: Mismatch-count distributions for $n = 64$. Honest signing (blue) concentrates near its mean of 0.64 mismatches, far below the threshold, while a forger’s counts (red) follow a fair-coin distribution centered at 32. The shaded accept region ends at $\lfloor n\tau \rfloor = 13$; the forger’s left tail inside it has total mass $P_{\text{forge}} = 9.4048 \times 10^{-7}$, the value derived in Section 5.4. Both curves are exact binomial distributions with the per-check probabilities from Section 5.2.

5.4 Forgery Bound P_{forge}

Where does the factor $1/2$ come from? There are two cases. If no session record exists, Bob’s side of the Bell pair reduces to the maximally mixed state $I/2$, and every measurement of $I/2$ is a fair coin regardless of basis, so her per-check success is exactly $1/2$. If a session record exists but the claimed syndrome is wrong and the attacker cannot see the true one, the net operator applied to the message state is the claimed correction composed with a uniformly random true one, which is uniform over the full Pauli group $\{I, X, Y, Z\}$; for any Pauli eigenstate, exactly two of these (the identity and the basis operator) leave the measurement outcome unchanged, so success is again $1/2$. Only in the special case where the attacker knows the true syndrome but cannot replay it (net uniform over the three non-identity operators) does the per-check success fall to $1/3$. The worst case across all cases is $1/2$. With M the number of mismatches a forger would suffer, M follows $\text{Binomial}(n, 1/2)$ and the verifier accepts only when $M \leq \lfloor n\tau \rfloor$. The forgery acceptance probability is the binomial tail:

$$P_{\text{forge}} = \sum_{k=0}^{\lfloor n\tau \rfloor} \binom{n}{k} \left(\frac{1}{2}\right)^n \quad (5)$$

Example Calculation:

- Allowed mismatches: $\lfloor 64 \times 0.2097 \rfloor = 13$.
- $P_{\text{forge}} = 2^{-64} \sum_{k=0}^{13} \binom{64}{k}$.
- Result: $P_{\text{forge}} = 9.4048 \times 10^{-7}$. Under the unentangled-adversary model a forger succeeds roughly once in a million attempts.
- At $n = 128$: $\lfloor 128 \times 0.1541 \rfloor = 19$ and $P_{\text{forge}} = 7.7792 \times 10^{-17}$.

Doubling n shrinks the margin by $\sqrt{2}$ and improves the forgery bound by ten orders of magnitude. That scaling holds under either threshold choice in the table above (about 1.2×10^{10} as built and 1.7×10^{10} tightened). The exact sum is computed in `detect/tau.py` with SciPy’s binomial CDF, not hardcoded.

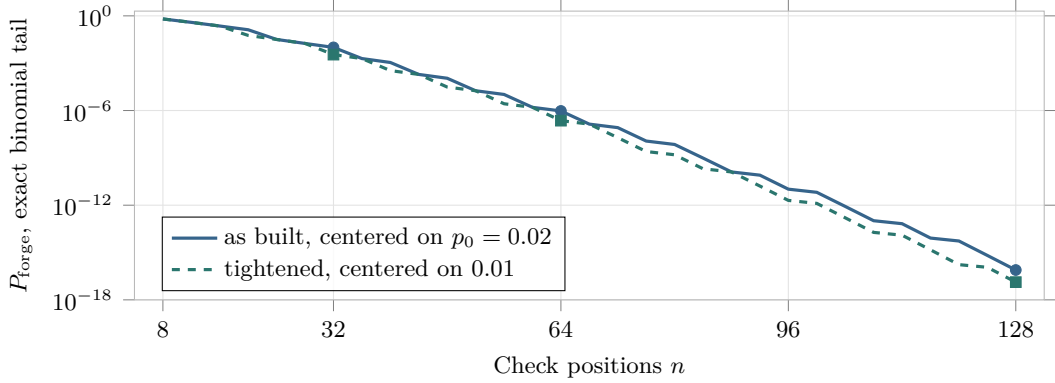


Figure 2: The forgery bound against the number of check positions n , evaluated with the exact binomial CDF at the Hoeffding thresholds, under the unentangled-adversary model. Solid: thresholds as built, centered on the noise floor $p_0 = 0.02$. Dashed: the tightened alternative centered on the honest mean 0.01 from the trade-off table above. Markers: the three operating presets. The exponential decay matches the stated ten-orders improvement from $n = 64$ to $n = 128$.

The $n = 32$ row in Section 8 is the downgrade floor: any bundle claiming fewer than 32 positions is rejected outright as parameter tampering. At the floor the forgery bound is only about 1%, which is why the NORMAL preset operates at $n = 64$ and why downgrade attempts are treated as an attack rather than a legitimate preset.

5.5 Message Binding

The expected bits come from the payload hash, so the signature is bound to the transaction. If an attacker changes the amount after signing, the hash changes, roughly half of the 256 hash bits flip, and the syndromes no longer match the expected bits. Each affected check then mismatches with probability $1/2$. This argument rests on the standard heuristic model that SHA-256 digest differences propagate as independent random bit flips, so the affected checks behave as independent Bernoulli($1/2$) variables and the tail bound of Section 5.4 applies. Avalanche behavior is a design property of SHA-256 backed by extensive empirical testing, not a theorem [7].

Example Calculation:

- Signed transaction: alice sends 100.00 INR to bob.
- Attacker edits the amount to 1000.00 INR.
- SHA-256 avalanche: about half of the 256 hash bits change (137 in one live run; the exact count depends on the serialized payload, the ratio stays near $1/2$).
- Mismatch rate jumps to $\hat{p} \approx 0.5$, which is far above $\tau = 0.2097$.
- Result: rejected. The SOC labels it FORGERY or PARAM_TAMPER.

This is covered by the payload-tamper test in `tests/test_attacks.py`.

6 Channel and Session Security

6.1 AES-GCM Wrap

The syndrome bits are wrapped with AES-256-GCM under a key derived from the BB84 session key through HKDF-SHA256 [9] (`ppqc/protect.py`). The GCM tag is 16 bytes, so a single blind tag guess succeeds with probability about 2^{-128} . The overall authentication guarantee also depends on nonce discipline and on the total volume of queries and ciphertext under one key; the standard GCM multi-query caveats apply [8]. Any corruption of the correction bits in transit fails the tag check and is classified as a channel attack. This layer is real cryptography from the `cryptography` library.

6.2 BB84 QBER Estimation

The session key itself is negotiated over a BB84 simulation (`qkd/bb84.py`). The honest baseline QBER is 2.5%. When Eve intercepts, she chooses the wrong basis with probability 1/2 and collapses the state; Bob then measures an error with probability 1/2. The QBER contribution of Eve alone, per intercepted qubit, before the baseline is added, is therefore:

$$\text{QBER}_{\text{Eve}} = \frac{1}{2} \times \frac{1}{2} \times p_e = \frac{p_e}{4} \quad (6)$$

Where:

- p_e = fraction of qubits Eve intercepts

The implementation applies the two noise sources sequentially, Eve’s collapse first and then the depolarizing baseline on top, so to first order the total observed QBER is the sum of the two. The value is estimated on a 64-bit sample of the sifted key, so measured QBER fluctuates around its expectation.

Example Calculation:

- Honest link: QBER $\approx$ 2.5%, stage Q0 (healthy).
- Partial tap ($p_e = 0.5$): Eve contributes 12.5%; with the additive baseline the total is about 15%, stage Q3.
- The channel-attack path models full interception ($p_e = 1.0$): the idealized intercept-resend value is 25% with no other channel noise; with the additive baseline the expectation is about 27.5%, and the 64-bit estimate typically lands in the 16 to 25% band, so the stage reads Q3/Q4 and the pipeline escalates to S4 full lock.

6.3 Replay Protection

Every bundle carries a UUID4 nonce with 122 random bits [10]. The classifier keeps a persistent nonce cache backed by SQLite (`shorlynot_nonces.db`), so accepted nonces survive service restarts. A bundle whose nonce was seen before is classified as replay and never reaches the statistical test.

6.4 Trust Boundaries

The statistical machinery assumes the classical layers are intact, and it is worth saying where that assumption sits. In a production deployment the session store plays the role of a key-management system: its enrollment writes would need authenticated key ceremonies (the prototype seeds it internally at signing time), the SQLite files would need filesystem-level protection, and the AES/HKDF key material would need rotation. Anyone with read or write access to the session store or the nonce

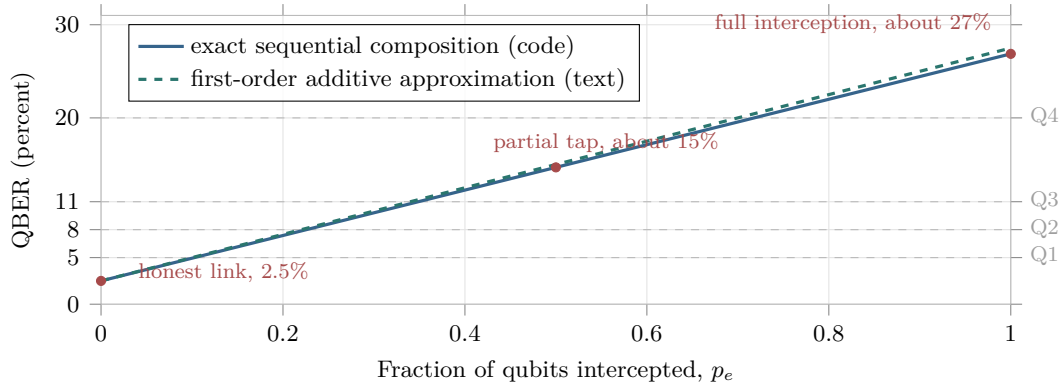


Figure 3: QBER against the fraction p_e of qubits Eve intercepts. Solid: the exact sequential composition applied in `qkd/bb84.py`, with Eve’s collapse first and baseline noise applied on top. Dashed: the first-order additive approximation used in the text; the two differ by at most 0.7 percentage points. The horizontal lines mark the Q-stage boundaries of the stage machine; Q0 covers the band below 5%. A partial tap lands in Q3 and full interception lands in Q4.

cache bypasses every check described in this document; the Hoeffding and binomial results do not apply inside the classical data layer. None of this is hidden: the repository documents the same boundaries in `KNOWN_LIMITATIONS.md`. For a bank deployment this is the actual attack surface: the tail bounds of Section 5 are the easy part of the security story, and protecting the store is the hard part.

7 Threat Classification and Response

The Q-STDF classifier (`detect/classifier.py`) is a fixed non-ML ladder with no training data, no model weights, and no drift. It checks, in order:

1. Unauthorized verifier: the caller is not in the verifier set.
2. Impersonation: the key id does not match the claimed signer’s registry binding.
3. Replay: the nonce is already in the cache.
4. Parameter tampering: fewer than 32 check positions or truncated arrays.
5. Channel: the AES-GCM tag fails or the QBER is elevated.
6. Forgery: $\hat{p} > \tau$.
7. Otherwise the transfer is OK.

The ladder is blind. The attack endpoints only build malicious bundles and submit them through the same verify path as honest transfers. No attack label is ever read by the classifier. The response side is the stage machine (`stages/state_machine.py`):

Stage	Meaning	Enforcement	Trigger
S0	Normal	Full service	OK verdicts
S1	Soft anomaly	Warn, log to SOC	Watch-level QBER
S2	Threat detected	Block actor’s new transfers	Forgery, impersonation
S3	Pattern attack	Read-only history, session reset	Replay, unauthorized verifier, repeated hits
S4	Critical	Full bank lock	Channel, parameter tampering, Q3/Q4, multi-hit

The lock is scope-aware. A single forgery sets `lock_scope` to account and quarantines only the actor who triggered it; other users keep transacting. The global lock is reached only on sustained or correlated evidence, channel compromise, or parameter downgrade. This prevents a one-request denial of service. The Q stages for link health are Q0 (below 5%), Q1 (5 to 8%), Q2 (8 to 11%), Q3 (11 to 20%) and Q4 (20% and above).

8 Monitoring and Dashboard

The SOC dashboard (`bank/app/templates/soc.html`) shows the live state of the engine: the current τ , p_0 , n and δ ; the last verdict with its mismatch rate; the S and Q stages; the quarantined accounts; and the recent event log. The bank portal displays the same parameters on every transfer screen, so the numbers a judge sees are the numbers the verifier actually used.

Metrics on the dashboard are computed from real telemetry collected during pipeline runs (accept rates, sign and verify latencies, forgery counts). There are no hardcoded dashboard values. The full benchmark table, generated by the analysis suite (`analysis/benchmark.py`), is in `docs/BENCHMARKS.md`:

n	τ	P_{forge} theoretical	Forgery successes (empirical, 100 runs)	Honest accept	Latency
32	0.2882	1.0031×10^{-2}	2 / 100	100.0%	71.6 ms
64	0.2097	9.4048×10^{-7}	0 / 100	100.0%	90.7 ms
128	0.1541	7.7792×10^{-17}	0 / 100	100.0%	178.4 ms

The empirical column is a Monte Carlo sanity check: 100 random-forgery runs per row, counting how many bundles pass the threshold. At $n = 32$ the theoretical rate predicts about one success per 100 runs; the recorded run observed 2, a separate run of the same suite observed 0, and both are consistent with the bound. At $n = 64$ the expected count is about 9.4×10^{-5} per 100 runs, so observing 0 is consistent with the bound but cannot verify it, and the same holds at $n = 128$. The theoretical column is the security claim. The two committed artifacts (`docs/BENCHMARKS.md` and `docs/benchmarks_summary.csv`) come from different runs of the same suite, which is why the $n = 32$ empirical cell differs between them.

The pipeline latency includes 64 actual Qiskit Aer simulation runs. Verification alone is an analytic Born-rule evaluation and runs in about 1 ms per transaction.

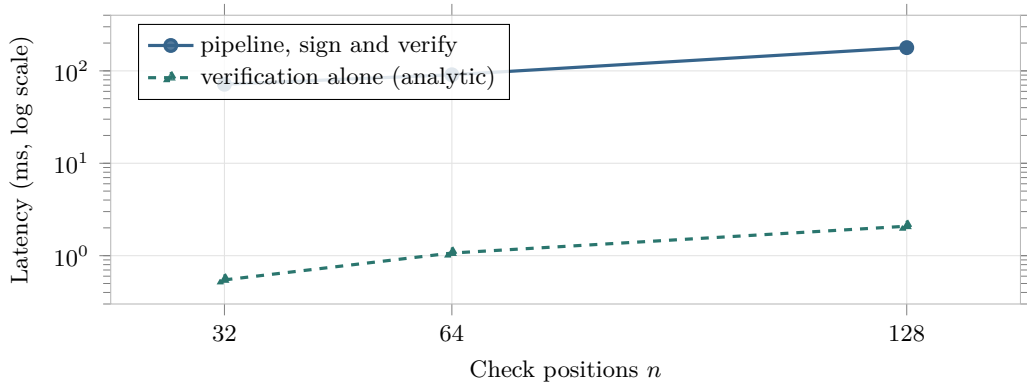


Figure 4: Measured latency from the analysis suite (`analysis/benchmark.py`) on the development machine at the three operating presets. Pipeline latency (solid) includes the sign circuits and dominates; verification alone (dashed) is the analytic evaluation, near 1 ms at $n = 64$. Each series plots the three rows of the benchmark table in this section.

The test suite reports 45 passing tests including real-circuit tests (`test_real_circuits.py`) and the payload-tamper test.

Verifying the 10^{-17} tail directly would need more runs than exist in the universe, which is why the theoretical column is the claim and the empirical column is only a guard against gross mis-specification.

9 Demo Walkthrough

This section walks through the two-minute fit test as it runs on a laptop. The same sequence is automated in `scripts/demo_attacks.py`. The attacks are scripted by the project’s own injectors and match the PS attack taxonomy (forgery, impersonation, replay, channel, parameter tampering, unauthorized verifier). The demo therefore validates the response ladder against anticipated attacks; it is not a claim of detecting novel adversarial discovery.

9.1 Case Study 1: Honest Transfer

- Overview: alice signs a transfer of 5000.00 INR to bob.
- Data: the engine runs 64 teleportation circuits, wraps the syndromes, and verifies. The observed mismatch rate is 0.00 against $\tau = 0.2097$.
- Result: accepted, stage stays S0, the ledger commits the transfer.

9.2 Case Study 2: Forgery and Account Quarantine

- Overview: the SOC triggers the scripted forgery injector from alice.
- Data: the forged bundle has no matching session record, so the checks behave like fair coins and $\hat{p} \approx 0.5 > \tau$.
- Result: rejected and labelled FORGERY. The stage moves to S2 with `lock_scope` set to account. alice is quarantined; her next transfer is refused with a quarantine message, while bob still transacts normally.

9.3 Case Study 3: Channel Compromise and Full Lock

- Overview: the SOC triggers the scripted channel injector. The ciphertext is corrupted and the link is fully intercepted.
- Data: the AES-GCM tag fails, and the QBER jumps from the 2.5% baseline to the 16 to 25% band (Q3/Q4).
- Result: classified CHANNEL, stage S4, every user is redirected to the emergency lockdown console. The operator reviews the log and resets stages to S0/Q0 to resume service.

10 Conclusion

ShorlyNot closes the loop that classical signing leaves open: verify the signature evidence, name the threat, and respond at the right scope. The contribution is at the systems layer, not in new cryptography: an existing QDS verification pattern implemented faithfully per the problem statement’s specification, with a graded, blast-radius-aware incident-response design built on top. The threshold $\tau = 0.2097$ and the forgery bound $P_{\text{forge}} = 9.4048 \times 10^{-7}$ are derived in Sections 5.3 and 5.4 under the unentangled-adversary model, and their limits are stated as plainly as their strengths. The simulation is stated openly, the cryptography at the edges is real, and the enforcement logic is backend-agnostic, so moving from Aer to hardware changes the noise statistics but not the design. Within that honest scope the system is a complete, testable answer to Problem Statement 26141.

11 Future Work

The name references Shor’s algorithm. The delivered layer rests on symmetric cryptography and statistical thresholds, which Shor’s algorithm does not attack; the post-quantum public-key upgrade is the ML-DSA item below.

- Run one small job on real IBM hardware and show the backend switch on the SOC, validating the noise-recalibration claim.
- Add multi-party QDS signatures (one-to-many and network settings).
- Integrate ML-DSA (FIPS 204) beside the QDS layer, answering the who-signed question that the tamper-evidence layer deliberately leaves open.
- Re-estimate p_0 continuously from live mismatch statistics.

References

References

- [1] D. Gottesman and I. Chuang, *Quantum digital signatures*, arXiv:quant-ph/0105032 (2001).
- [2] P. Wallden, V. Dunjko, A. Kent, and E. Andersson, *Quantum digital signatures with quantum-key-distribution components*, Phys. Rev. A 91, 042304 (2015).
- [3] W. Hoeffding, *Probability inequalities for sums of bounded random variables*, J. Amer. Statist. Assoc. 58, 13–30 (1963).
- [4] C. H. Bennett and G. Brassard, *Quantum cryptography: public key distribution and coin tossing*, Proc. IEEE Int. Conf. on Computers, Systems and Signal Processing (1984).
- [5] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge University Press (2010).
- [6] NIST, FIPS 203 (ML-KEM) and FIPS 204 (ML-DSA).
- [7] NIST, *Secure Hash Standard (SHS)*, FIPS PUB 180-4 (2015).
- [8] M. Dworkin, *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, NIST SP 800-38D (2007).
- [9] H. Krawczyk and P. Eronen, *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*, RFC 5869 (2010).
- [10] K. Davis, B. Peabody, and P. Leach, *Universally Unique IDentifiers (UUIDs)*, RFC 9562 (2024).
- [11] Qiskit contributors, *Qiskit: An Open-Source Framework for Quantum Computing*, doi:10.5281/zenodo.2573505 (2023).